🗎 URL Shortener Service(C++)

1. Introduction

This project implements a simple URL Shortener Service using C++. The system allows users to convert long URLs into shorter, more manageable links and retrieve the original URLs when needed. The application is CLI-based and uses only the C++ standard library.

2. Problem Statement

Long URLs are difficult to share, read, and manage. This project aims to provide a simple solution by generating shorter, human-readable URLs that map to the original links

3. Objectives

- Convert long URLs into short URLs
- Retrieve original URLs from short ones
- Ensure uniqueness of generated links
- Store data persistently using files
- Validate user input

4. System Requirements

4.1 Functional Requirements

- Shorten a long URL
- Resolve a short URL
- Store mappings in a file
- Avoid duplicate entries
- Validate input

4.2 Non-Functional Requirements

- Implemented in C++
- No external libraries
- CLI-based
- Modular and clean code

5. System Design Overview

The system follows a modular layered architecture:
- Presentation Layer (CLI)
- Business Logic Layer (URLShortener)
- Utility Layer (Validator + Generator)
- Persistence Layer (Storage)

Components:

- Code Generator
- Storage Manager
- URL Shortener (Main Logic)
- CLI Interface

6. Use Case Description

Actor:

- User

Use Cases:

1. Enter a long URL to shorten
2. Enter a short URL to retrieve original
3. Exit system

7. Data Structures

The system uses:

- `unordered_map<string, string>`

Purpose:

- Store mapping between short codes and long URLs
- Provide fast lookup (O(1))

8. Class Design (OOP)

Class: URLValidator

- `bool isValid(string url)`

Class: CodeGenerator

- `string generate(string url)`

Class: Storage

- `void save(string shortCode, string longURL)`
- `string find(string shortCode)`
- `void load()`

Class: URLShortener

- `string shortenURL(string longURL)`
- `string resolveURL(string shortCode)`

9. Class Diagram

URLShortener --> URLValidator
URLShortener --> CodeGenerator
URLShortener --> Storage
Storage → File System (Persistent Storage Layer)

10. System Flow

Shortening Process:

1. User inputs long URL
2. Validate URL
3. Check if it already exists
4. Generate short code
5. Save mapping
6. Return short URL

Resolving Process:

1. User inputs short URL
2. Extract short code
3. Search in storage
4. Return original URL

11. Storage Design

Data is stored in a text file.

Format:

short_code | long_url

The short code format is dynamically generated and may vary based on hashing and domain parsing.

Example:

yt/abc | https://youtube.com/watch?v=abc

gh/123 | https://github.com/project/issues/123

12. Validation Rules

- URL must start with `http://` or `https://`
- Must not be empty
- Must contain a valid domain
- must not exceed reasonable length
- no spaces allowed

13. Design Decisions

The system ensures separation of concerns to improve maintainability and scalability

- Used `unordered_map` for fast lookup
- Used file storage for simplicity
- Used OOP to separate concerns
- CLI used to reduce complexity

14. Testing Plan

Test cases include:

- Valid URL → shortened successfully
- Invalid URL → error message
- Duplicate URL → same short code
- Unknown short URL → error message

15. Build & Run Instructions

Compile:

```
g++ main.cpp URLShortener.cpp URLValidator.cpp CodeGenerator.cpp Storage.cpp -o app
```

Run:

```
./app       (Linux/Mac)
.\app.exe    (Windows)
```